# Sod tube tutoriel

## Jérémy Fensch - March 2026

# 1 Getting started

WARNING: Document created for hands-on the CBP machines. Adapt it to your own computing environment.

## 1.1 First compilation

Let's start by cloning RAMSES in the desired directory:
$ cd YOUR_WORK_DIRECTORY # optional
$ git clone https://github.com/ramses-organisation/ramses

Then for a quick compilation:
$ cd ramses/bin
$ make

You now have an executable for RAMSES (ramses1d). This is the default executable, it is not usable for every setup.

## 1.2 First simulation

Now, create a folder for your first simulation, and copy there the following files from the CBP:

$ cd ../.. or $ cd YOUR_WORK_DIRECTORY
$ mkdir Sod_tube
$ cd Sod_tube
$ cp -r /home/jfensch/Sod/* .

In this folder there are three files and a directory:

- tube1.nml: file containing the initial conditions
- read_log.py: a file creating plots from the logs
- shocktubecalc: a python package calculating the sod tube solution
- run_ramses.sh: a wrapper, launching the simulation and the creation of a movie: sod.mp4

You can launch the simulation only simply by doing:
$ ../ramses/bin/ramses1d tube1d.nml

But all the results are in the created log file, we need a bit of work to visualize that. I created a post-processing routine that reads the outputs and create a movie:

$ sh run$_r amses_m ovie.sh$

You should obtain a .mp4 movie showing the evolution of the density, velocity and pressure profiles with time: each black square is a cell. In the background, the analytical solution is in dark blue and the refinement level profile in light blue, as in Fig. 1. As it takes $\simeq$ 1 min to create the movie, you may want to only get the last snapshot. To do that, you can run instead:

$ sh run$_r amses_p lot.sh$

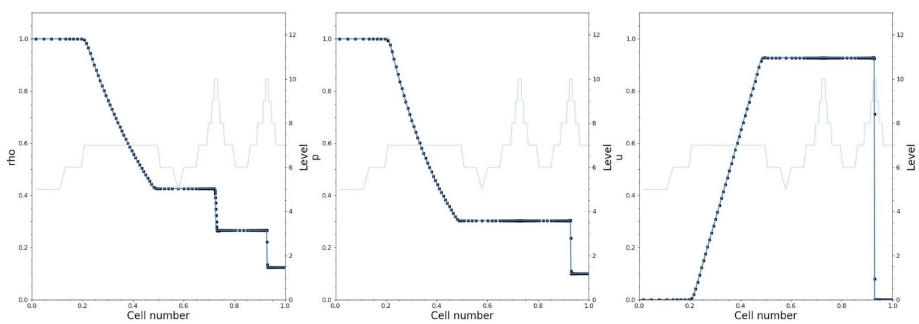


Figure 1: Result of the first simulation.

## 2 Playing with the parameters

Now you can see what are the effects of changing the parameters directly in the namelist. For instance:

- levelmax: that is the highest refinement level authorized. No need to increase it above 10 (memory cost and you are computing in the $HOME!)
- courant_factor: see Fig. 2.
- slope_type: type of slope limiter used in the Godunov scheme for the piecewise linear reconstruction: slope_type = 0 : First order scheme, slope_type = 1 : MinMod limiter, slope_type=2: MonCen limiter. slope_type=3: Multi-dimensional MonCen limiter. In 1D runs only, it is also possible to choose: slope_type=4: Superbee limiter slope_type=5: Ultrabee limiter
- riemann: type of solver: Possible choices are 'exact', 'acoustic', 'llf', 'hll' or 'hllc'

- &REFINE_PARAMS: where you define the refinement strategy: here gradients on density, velocity and pressure.

Less interesting for the class, but for your interest:

- &INIT_PARAMS: you can change the initial conditions (would not match the analytical solution anymore of course)
- tout = 0.245 : end time of the simulation.
- interpol_var: 0 for conservative, 1 for primitive variables
- interpol_type : type of slope limiter used in the interpolation scheme for newly refined cells. interpol_type=0: Straight injection (1st order) interpol_type=1: MinMod limiter, interpol_type=2: MonCen limiter.

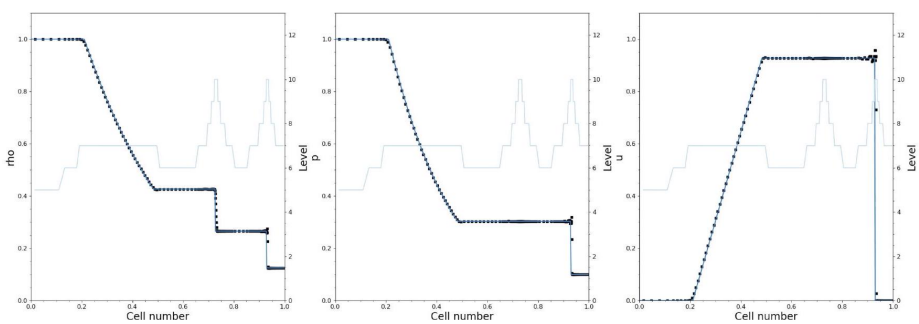


Figure 2: Result of a simulation with Courant factor = 1.1.